

Abstract

The Network Traffic Analysis using Machine Learning Algorithms project is designed to monitor, analyze, and detect anomalies in network traffic to enhance cybersecurity and network performance. By using machine learning, the system can learn from historical traffic patterns, detect previously unknown attacks, and provide automated, real-time network monitoring. This improves network security.

Network traffic analysis plays a crucial role in ensuring cybersecurity by monitoring, analyzing, and detecting suspicious activities within a network. With the rapid growth of network-based systems, traditional rule-based detection methods are often insufficient to handle complex and evolving cyber threats.

This project focuses on implementing an advanced network traffic analysis system using Automated Machine Learning (AML) techniques to improve detection accuracy and efficiency.

The proposed system collects network traffic data and preprocesses it to remove noise and irrelevant information. Using AML, the system automatically selects the best machine learning models, optimizes hyperparameters, and identifies patterns in network behavior. This enables the detection of anomalies such as unauthorized access, malware activity, and potential intrusions in real time.

The model is trained and tested on benchmark datasets to evaluate its performance based on metrics such as accuracy, precision, recall, and F1-score. The results demonstrate that AML-based approaches significantly enhance the detection capability compared to traditional methods while reducing manual effort in model selection and tuning.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
1	ABSTRACT	
	1.1 Introduction 1.2 About the Project 1.3 System Study 1.4 Problem Definition 1.5 Existing System 1.6 Proposed System 1.7 Advantages of Proposed System 1.8 Disadvantages of Proposed System	
2	PROJECT DESCRIPTION	
3	SYSTEM SPECIFICATIONS	
	3.1 Hardware Specification 3.2 Software Specification 3.3 Software Description	
4	DATABASE DESIGN	
5	SYSTEM DESIGN 5.1 Data Flow Diagrams	
6	SYSTEM IMPLEMENTATION	
	6.1 Testing and Implementation 6.2 Testing	
7	CHALLENGES FACED	
	7.1 Challenges Faced 7.2 Testing 7.3 Outcome	
8	SOURCE CODE AND IMPLEMENTATION	
	9.1 Programming Language 9.1.1 Programming Language Used 9.2 Libraries and Modules 9.3 Dataset Loading Module 9.4 Data Preprocessing Module 9.5 Feature Selection Module 9.6 Model Training Module 9.7 Model Testing and Evaluation 9.9 Output Generation	
10	CONCLUSION AND FUTURE ENHANCEMENTS	
	10.1 Conclusion 10.2 Future Enhancements	
11	BIBLIOGRAPHY	

1. Introduction

1.1 About the Project

- Network security has become a major concern due to the rapid growth of computer networks and internet usage.
- Cyber attacks such as intrusion, malware, denial-of-service attacks, and data theft are increasing day by day.
- Traditional security mechanisms are not sufficient to detect modern and complex attacks.
- This project, “Network Traffic Analysis Using Machine Learning”, focuses on analyzing network traffic data to identify malicious activities and vulnerabilities.
- Machine learning algorithms are used to classify network traffic as normal or abnormal based on learned patterns
- The system helps in improving detection accuracy and reducing human effort.

2. System Study

2.1 Problem Definition

Traditional network security systems depend on predefined rules and signatures. These systems fail to detect new and unknown attacks.

- Manual monitoring of network traffic is time-consuming and error-prone.
- Hence, there is a need for an intelligent system that can automatically analyze network traffic and detect vulnerabilities.

2.2 Existing System

- The existing system uses firewalls, intrusion detection systems (IDS), and rule-based monitoring techniques. These systems rely on static rules and require frequent updates. They are ineffective against zero-day attacks and advanced persistent threats.

2.3 Proposed System

- The proposed system uses machine learning algorithms to analyze network traffic patterns.
- The system learns from historical data and detects abnormal behavior automatically.
- It improves detection accuracy and adapts to new attack patterns.
- When new network traffic enters the system, the trained machine learning model analyzes the data and predicts whether the traffic is normal or malicious.

- If malicious activity is detected, the system alerts the administrator so that appropriate security actions can be taken.

2.4 Advantages of Proposed System

- Automatic detection of malicious network traffic
- Faster analysis compared to manual monitoring
- Improved network security
- Helps detect cyber attacks early

2.5 Disadvantages of Proposed System

- The system collects network traffic data from the network using packet capturing tools or datasets.
- The collected data is preprocessed and important features are extracted such as IP address, protocol, and packet size.
- The system detects normal and malicious traffic and alerts.
- The administrator if any suspicious activity is found.

3.Project Description

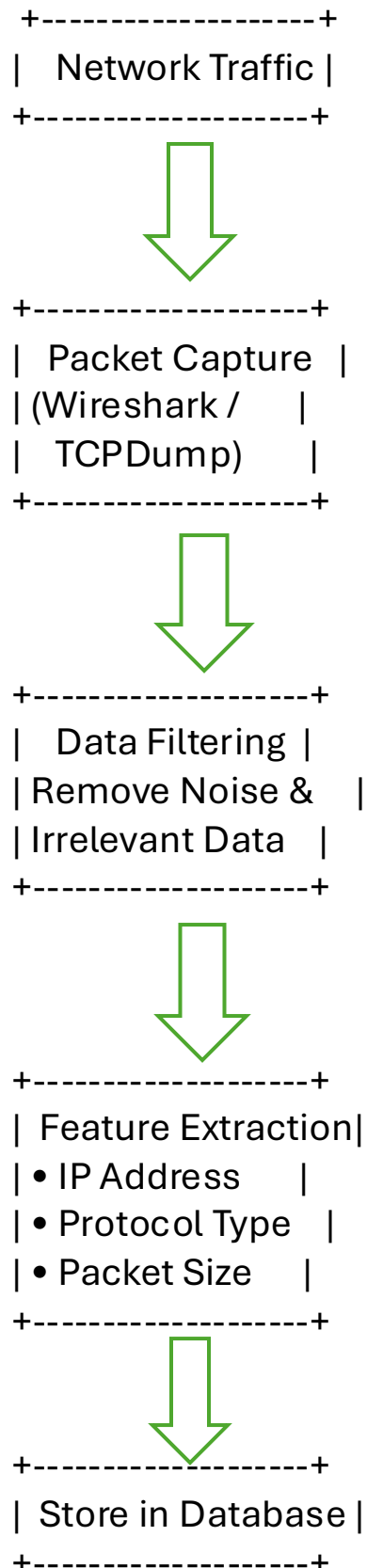
PROJECT DESCRIPTION

- The project collects network traffic data from datasets or live network environments.
- The data is pre-processed and important features are extracted.
- Machine learning models are trained using labeled data.

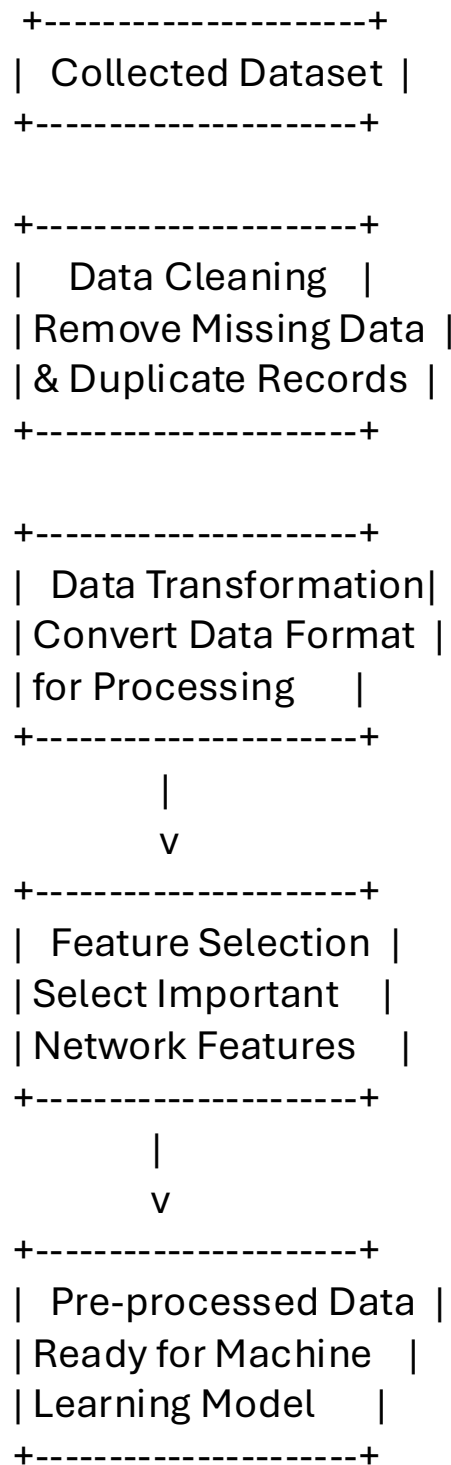
The system architecture consists of:

- Data collection module
- Data preprocessing module
- Machine learning model
- Classification and alert module

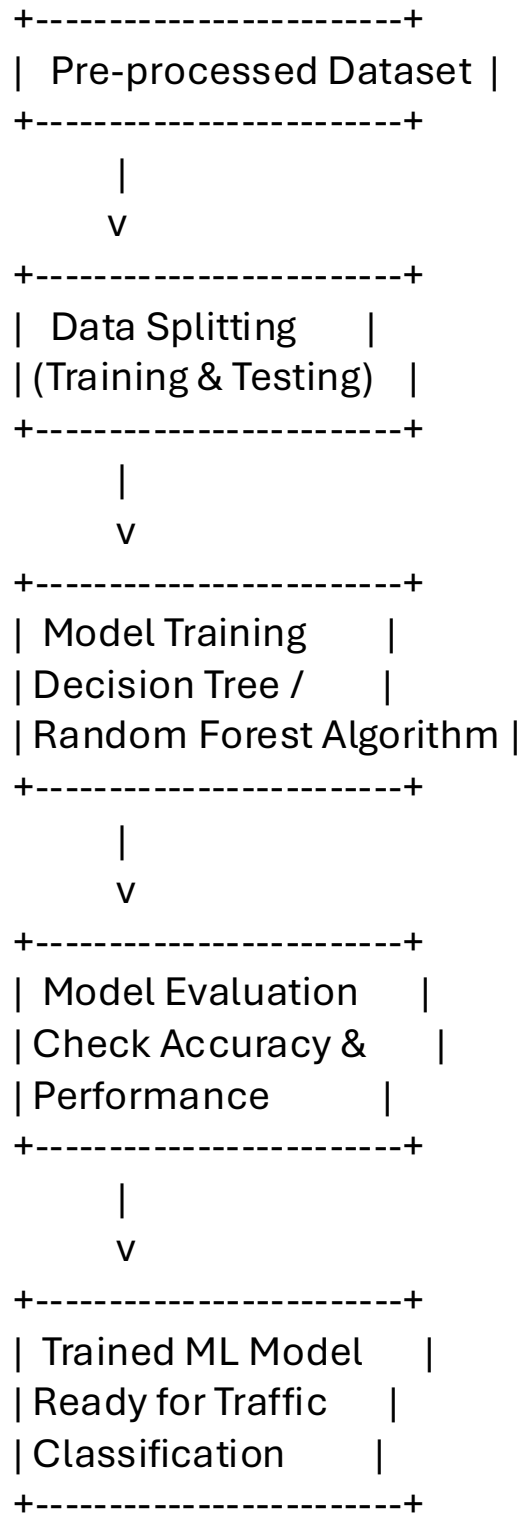
Data Collection Modules



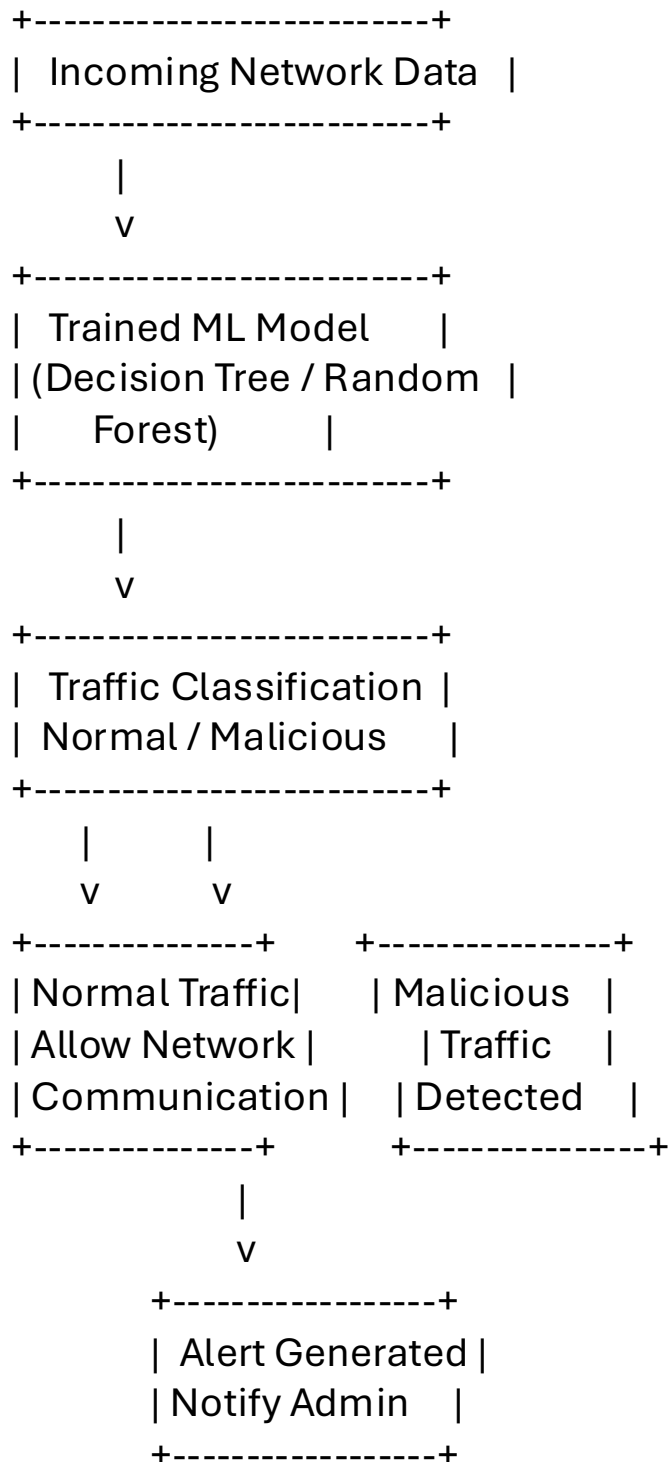
Data Preprocessing Modules



ML Models



Classification And Alert Models



4.System Specification

4.1 Hardware Specification;

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Hard Disk: 20 GB free space
- Network Interface Card

4.2 Software Specification;

- Operating System: Windows / Linux
- Programming Language: Python
- Tools: Jupyter Notebook / VS Code
- Libraries: NumPy, Pandas, Scikit-learn, Matplotlib

4.3 Software Description – Network Traffic Analysis Using (ML)

The Network Traffic Analysis Using Machine Learning system is developed using modern software tools and libraries that help capture, process, analyze, and classify network traffic data. The software components work together to detect abnormal or malicious network activities automatically.

1. Operating System

- The project can run on operating systems such as Windows or Linux.
- These platforms provide the required environment to run Python programs and network analysis tools.

2. Programming Language

The main programming language used in this project is Python. Python is widely used in machine learning and cybersecurity projects because it is simple, powerful, and supports many useful libraries for data analysis and network monitoring.

3. Development Tools

- The project is developed using tools such as Jupyter Notebook or Visual Studio Code (VS Code).
- Jupyter Notebook is useful for writing and testing machine learning models interactively.
- VS Code provides a powerful coding environment with debugging and extension support.

4. Python Libraries

- Several Python libraries are used to implement the system:
- NumPy – Used for numerical computations and handling arrays.
- Pandas – Used for data collection, cleaning, and preprocessing of network traffic datasets.
- Scikit-learn-Used to build machine learning models such as Decision tree and Random forest for traffic classification.

5. Network Analysis Tool;

- Tools like Wireshark can be used to capture network packets and collect traffic datasets.
- These datasets are then used for training and testing the machine learning models.

6. Dataset Handling;

- The captured network traffic data is stored in formats such as CSV files.
- Which are processed using Python libraries for further analysis and classification.

5.Database Design

1. Introduction Of Database

- Design is an important part of the “Network Traffic Analysis Using Machine Learning” system.
- It stores network traffic data, processed features, user details, and classification results.
- The database helps the system efficiently manage and analyze network traffic to detect malicious activities.

2. Database Purpose

The main purpose of the database is to store captured network traffic data store pre-processed datasets used for machine learning save model prediction results maintain alerts and logs for suspicious traffic manage user and system information.

Database Tables

- User Table
- Network Traffic Table
- Pre-processed Data Table
- ML Prediction Table
- Alert Table

User Table

User Table

Field Name	Data Type	Description
User_ID	INT (PK)	Unique ID for each user
Username	VARCHAR	User login name
Password	VARCHAR	User password
Email	VARCHAR	User email address
Role	VARCHAR	User role (Admin/User)

Network Traffic Table

Network Traffic Table

Field Name	Data Type	Description
Traffic_ID	INT (PK)	Unique traffic ID
Source_IP	VARCHAR	Source IP address
Destination_IP	VARCHAR	Destination IP address
Protocol	VARCHAR	TCP/UDP/ICMP protocol
Packet_Size	INT	Size of packet
Timestamp	DATETIME	Time of capture

Pre-processed Data Table

Preprocessed Data Table

Field Name	Data Type	Description
Data_ID	INT (PK)	Preprocessed data ID
Traffic_ID	INT	Reference traffic ID
Feature_Vector	TEXT	Extracted features
Label	VARCHAR	Normal or Malicious

ML Prediction Table

ML Prediction Table

Field Name	Data Type	Description
Prediction_ID	INT (PK)	Prediction record ID
Traffic_ID	INT	Related traffic
Model_Name	VARCHAR	ML model used
Prediction_Result	VARCHAR	Normal / Attack
Accuracy	FLOAT	Model confidence

Alert Table

Alert Table

Field Name	Data Type	Description
Alert_ID	INT (PK)	Alert record ID
Traffic_ID	INT	Related traffic
Alert_Type	VARCHAR	Type of attack
Severity	VARCHAR	Low / Medium / High
Alert_Time	DATETIME	Alert generated time

User Table;

- Manages system users
- This table stores the details of system users.

Network Table;

- This table stores captured network packets.

Preprocessed Data Table;

- Linked to network traffic
- Stores cleaned and processed data used for machine learning models.

Machine Learning Table;

- Stores the classification results from machine learning models.

Alert Table;

- Stores alerts generated when suspicious or malicious traffic is detected.

Advantages Of Database Design

Advantages of Database Design

Data Organization

- Database design organizes data into tables (like User, Network Traffic, Alerts), so the data is structured and easy to manage.

Reduced Data Redundancy

- Proper design avoids duplicate data. This saves storage space and improves efficiency.

Improved Data Consistency

- When data is stored correctly in a database, updates happen in one place, so the information remains accurate and consistent.

Better Data Security;

- Database systems allow access control (user permissions), which helps protect sensitive data.

Faster Data Retrieval;

- With proper indexing and structure, databases allow quick searching and retrieval of information.

Easy Data Maintenance;

- It becomes easier to update, delete, or modify data without affecting other parts of the system.

Supports Data Analysis;

- Well-designed databases help in analyzing data easily, which is useful for projects like Network Traffic Analysis using Machine Learning .

Scalability;

- A good database design can handle increasing data as the system grows.

Disadvantages Of Database Design

1. Large Storage Requirement;

- Network traffic generates a huge amount of data every second.
- Storing all packets and logs in the database requires large storage space
-

2. High Processing Time;

- When the database contains millions of traffic records.
- Retrieving and processing the data may take more time, which can slow down the system.

3. Data Privacy Issues;

- Network traffic may contain sensitive information such as IP addresses and user activity.
- If the database is not properly secured, it may lead to privacy or security risks.

4. Complex Database Management;

- Managing and maintaining a large database.
- With multiple tables and relationships can be difficult and requires proper database administration.

5. Performance Issues;

- Continuous traffic collection and storage may cause database performance degradation.
- Especially if the system is not optimized.

6. Data Redundancy

- Possibility If the database design is not properly normalize.
- Duplicate traffic records may be stored, increasing storage usage.

6.System Design

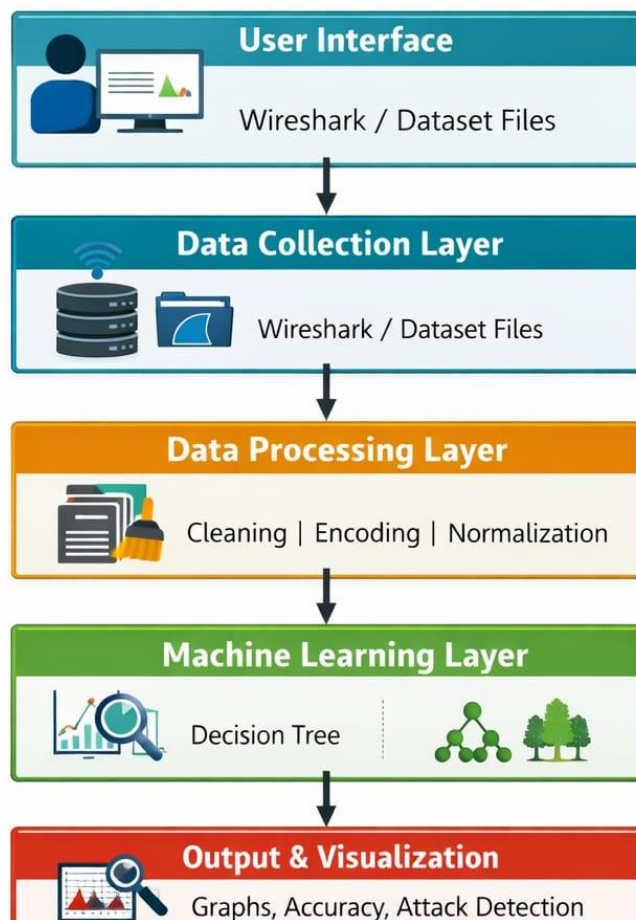
6.1 Data Flow Diagram

Network Traffic Analysis Using Machine Learning – Level 1 DFD



6.2 Architecture Diagram

Architecture for Network Traffic Analysis Using Machine Learning



Data Flow Diagram (DFD)

This diagram shows how network data is processed step-by-step using Machine Learning (ML) to detect whether traffic is normal or malicious.

1. Network Data (Input)

- The process starts with raw network data.
- This data comes from network devices like routers, servers, or firewalls.
- It includes information like IP address, packets, protocols, etc

2. Data Collection

- In this stage, the system captures network traffic data.
- Tools or sensors are used to monitor and collect real-time traffic.

Example: Packet sniffers (like Wireshark).

3. Data Preprocessing

- The collected data is usually dirty or unstructured.

So, it is cleaned and formatted:

- Remove missing values
- Eliminate duplicates
- Convert data into proper format

This step makes data ready for analysis.

4. Feature Extraction

Important features are selected from the data.

Features are specific attributes like:

- Packet size
- Protocol type
- Duration of connection
- This reduces unnecessary data and improves accuracy.

5. ML Model (Training / Testing)

The processed data is given to a Machine Learning model. Two main steps:

Training: Model learns from past data

Testing: Model checks new data Algorithms like Decision Tree, Random Forest, etc. are used.

6. Traffic Classification

The trained model classifies traffic into:

- Normal Traffic
- Malicious Traffic This helps in identifying cyber attacks.

7. Output Result

Final results are generated:

- Alerts for malicious activity
- Reports or logs

This helps admins take action quickly.

Architecture Diagram

- This system is designed to monitor, analyze, and detect network attacks using machine learning.
- The architecture is divided into different layers, each performing a specific task.

This is the front-end part of the system. The user interacts with the system here.

Users can

1. Upload dataset files

- Capture live network traffic using Wireshark
- It acts as the entry point of the system.

2. Data Collection Layer

This layer collects the network traffic data.

Data can come from:

- Wireshark (live packet capture)
- Predefined dataset files
- The collected data is stored for further processing.

3. Data Processing Layer

Raw data is not directly useful, so it is processed here. Main steps

include:

- Cleaning → Removing errors or unwanted data
- Encoding → Converting text data into numeric form
- Normalization → Scaling values to a standard range

4. Machine Learning Layer

- This is the core part of the system.
- Machine learning algorithms are applied here.
- Example: Decision Tree algorithm is used to classify traffic. It analyzes whether traffic is:

Normal And Malicious / Attack

5. Output & Visualization

Final results are shown to the user.

Outputs include:

- Graphs
- Accuracy of the model
- Attack detection results

7. Testing And Implementation

7.1 Implementation

Implementation is the process of converting the system design into a working model.

In this project, Network Traffic Analysis is implemented using Python and Machine Learning algorithms to classify traffic as normal or malicious.

Step 1: Environment Setup

Install Python 3.x

Install required libraries:

- NumPy
- Pandas
- Scikit-learn
- Matplotlib
- Use IDE like Jupyter Notebook or VS Code
- Network packets captured using Wireshark (or dataset in CSV format)

Step 2:

Data Collection Traffic data is collected from:

Live network capture or Public datasets (like KDD, CICIDS)

The dataset contains:

Source IP Destination IP Port numbers Protocol Packet size Traffic label
(Normal / Attack)

Step 3:

Data Preprocessing Remove missing values Convert categorical values into numeric format Normalize numerical features Split dataset into:

- Training data (70–80%)
- Testing data (20–30%)

Step 4:

Model Training Machine Learning algorithms used:

- Decision Tree
- Random Forest Model is trained using training dataset.

Step 5:

Prediction and Classification Test data is given to trained model
Model predicts whether traffic is: Normal Malicious

Step 6:

Result Generation System displays: Accuracy score Confusion matrix
Classification report Graphical representation (optional)

7.2 Testing

- Testing ensures that the system works correctly and gives accurate classification results.
- Types of Testing Used Unit Testing Each module tested separately:
- Data loading
- Preprocessing
- Model training
- Prediction

Integration Testing

- Combined all modules
- Checked proper data flow between modules

System Testing

- Tested complete using dataset
- Verified final output accuracy
Performance

Performance Testing Checked:

- Model accuracy
- Processing time
- False positives and false negatives

Evaluation Metrics Used

- Accuracy
- Precision
- Recall
- F1-Score
- These metrics measure how well the model detects malicious traffic.

7.3 Challenges Faced

- Handling large dataset
- Data imbalance (more normal traffic than attacks)
- Overfitting in machine learning model
- Feature selection complexity

7.4 Final Outcome;

- The implemented system successfully
- Analyzes network traffic
- Extracts important features
- Classifies traffic using ML algorithms
- Detects malicious traffic with good accuracy

7.5 Working Process Screenshots

Data Collection; The system is capturing live network traffic

- Each row = one data packet
- It shows:
- Source IP (who is sending data)
- Destination IP (who is receiving)

-
- | No. | Time | Source | Port | Destination | Port | Protocol | Length | Load | Size | Info | Offset |
|---|-------|---------------|------|-------------|----------------------|----------|---------------------|----------------|------|------|--------|
| TC | 3TC06 | 192.168.1.105 | STCP | 408 | 23.217.138.110 | Probood | ACKK=86903 | 23.217.78.88.1 | 1100 | 443 | |
| TC | 3TC06 | 192.168.1.105 | TTCP | 448 | 23.217.138.111 | Protoed | ACKK=8692530 | 23.217.78.88.1 | 1100 | 443 | |
| TC | 3TC06 | 192.168.1.105 | YTPP | 486 | 23.23.788.38.8.37251 | Proteed | ACKK=88928 | 23.217.78.88.1 | 1100 | 443 | |
| FC | 3TC06 | 192.168.1.105 | STOP | 408 | 23.217.138.110 | Probood | ACKK=868338 | 23.217.78.88.1 | 1100 | 443 | |
| IC | 3TC06 | 192.168.1.103 | STOP | 408 | 23.217.138.111 | Proteed | ACKK=85808 | 23.217.78.88.1 | 1100 | 443 | |
| TC | 3TC06 | 192.168.1.105 | TTCP | 448 | 23.217.138.110 | Probood | ACK990 / ACK=882511 | 23.217.78.88.1 | 1100 | 448 | |
| BC | 3TC06 | 192.168.1.105 | TTCP | 448 | 23.217.88.110 | Protee | ACKK=38904 | 23.217.78.88.1 | 1100 | 448 | |
| SC | 3TC06 | 192.168.1.105 | TTCP | 448 | 23.217.88.111 | Protee | ACKK=38904 | 23.217.78.88.1 | 1100 | 444 | |
| CC | 3TC06 | 192.168.1.100 | TTCP | 408 | 23.23.788.38.8.37251 | Proteed | ACKK=88903 | 23.217.78.88.1 | 1100 | 443 | |
| CC | 3TC06 | 192.168.1.105 | TTCP | 448 | 23.217.138.110 | Proteed | ACKK=38903 | 23.217.78.88.1 | 1100 | 443 | |
| BC | 3TC06 | 192.168.1.106 | TTCP | 408 | 23.217.88.110 | Protee | ACKK=38904 | 23.217.78.88.1 | 1100 | 443 | |
| TC | 3TC06 | 192.168.1.105 | TTCP | 408 | 23.217.88.38.8.37251 | Proteed | ACK99 / ACK=882511 | 23.217.78.88.1 | 1100 | 443 | |
| CC | 3TC06 | 192.168.1.105 | TTCP | 408 | 23.217.238.110 | Proteed | ACKK=ACK=88931 | 23.217.78.88.1 | 1100 | 443 | |
| <pre> 1 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 2 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 3 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 4 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 5 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 6 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 7 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 8 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 9 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 10 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 11 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 12 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 13 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 14 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 15 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 16 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 17 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 18 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 19 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 20 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 21 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 22 00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 23 00000000: 00000000 00000000</pre> | | | | | | | | | | | |

Data Preprocesss;

Load Dataset Python;

```
data = pd.read_csv("traffic.csv")
```

- Network traffic data is loaded from a file

Remove Missing Values Python;

```
data = data.dropna()
```

- Removes empty or null values
- Ensures clean dataset

Normalize Data Python;

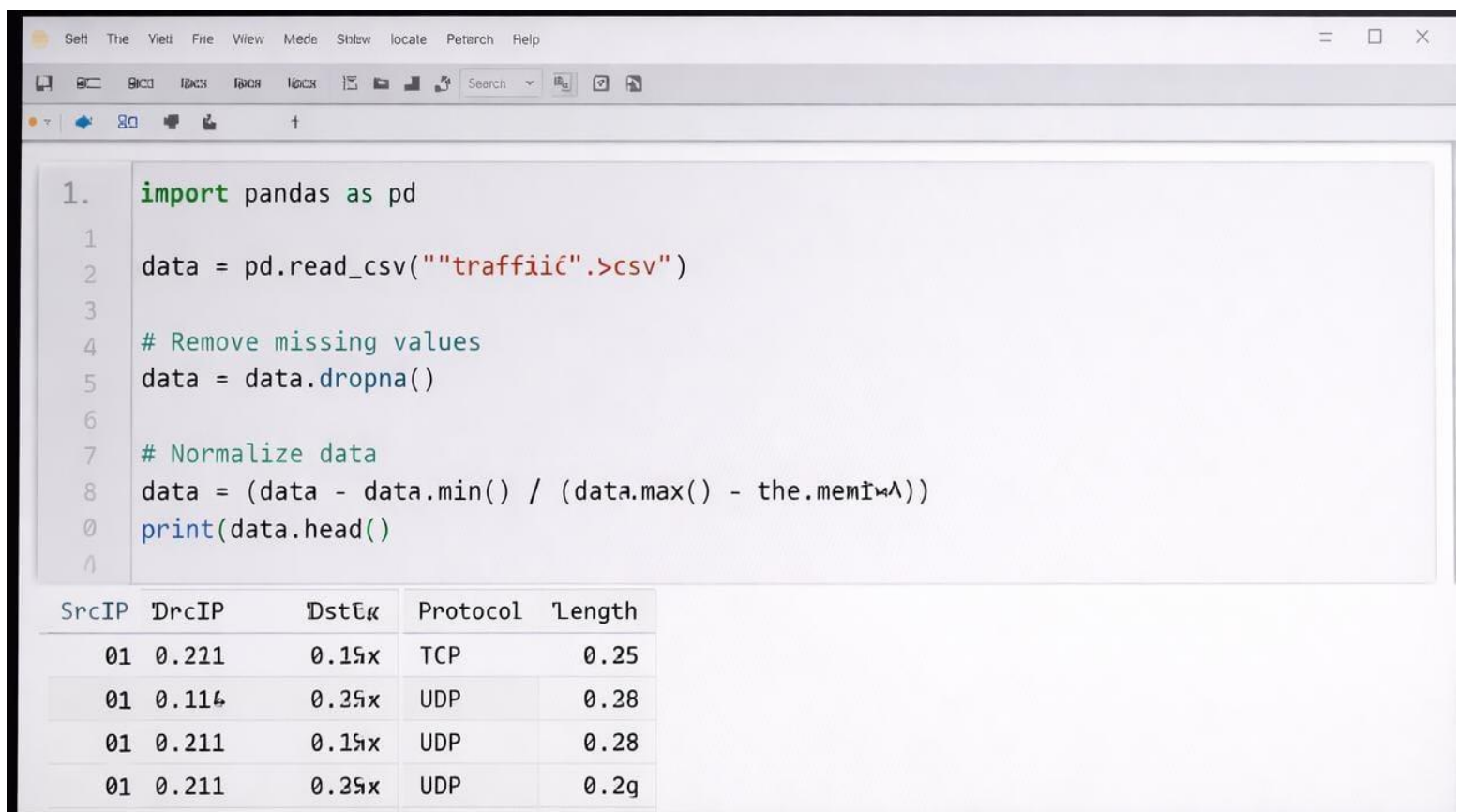
```
data = (data - data.min()) / (data.max() - data.min())
```

- Converts values into range 0 to 1
- Helps machine learning model work better

Display Output Python;

```
print(data.head())
```

- Shows first few rows of cleaned data



The screenshot shows a Jupyter Notebook interface. The code cell contains the following Python code:

```
1. import pandas as pd
1
2 data = pd.read_csv("traffii.csv")
3
4 # Remove missing values
5 data = data.dropna()
6
7 # Normalize data
8 data = (data - data.min()) / (data.max() - the.min())
9 print(data.head())
10
```

The output cell displays the first five rows of the cleaned and normalized dataset as a table:

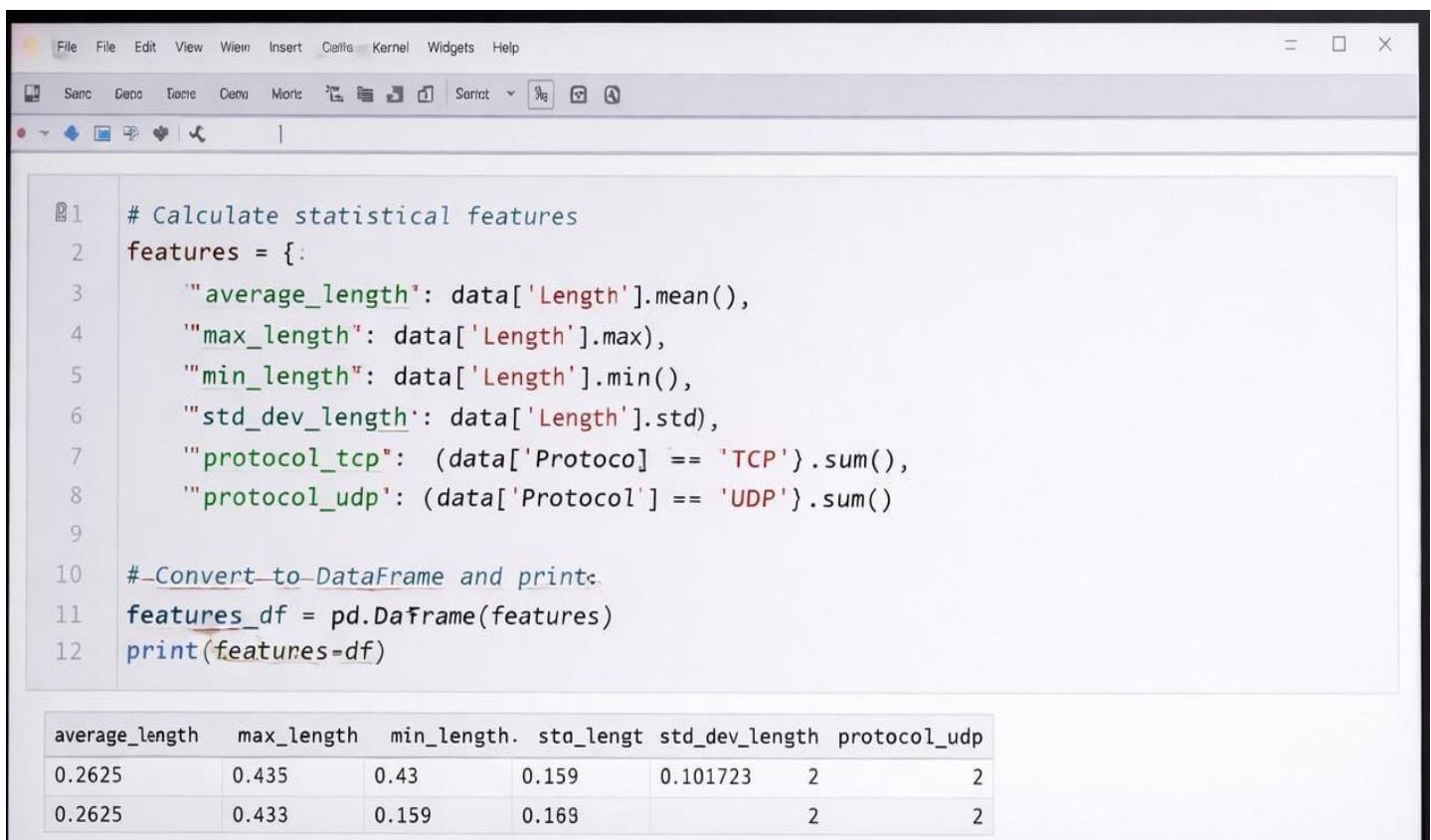
SrcIP	DrcIP	DstE	Protocol	Length
01	0.221	0.15x	TCP	0.25
01	0.114	0.25x	UDP	0.28
01	0.211	0.15x	UDP	0.28
01	0.211	0.25x	UDP	0.2q

Feature Extraction

The code is converting raw network data into useful features for Machine Learning.

- `average_length` → Average packet size
- `max_length` → Maximum packet size
- `min_length` → Minimum packet size
- `std_dev_length` → Variation in packet size
- `protocol_tcp` → Number of TCP packets
- `protocol_udp` → Number of UDP packets

These are called “features” (important inputs for ML model)



```
1 # Calculate statistical features
2 features = {
3     "average_length": data['Length'].mean(),
4     "max_length": data['Length'].max(),
5     "min_length": data['Length'].min(),
6     "std_dev_length": data['Length'].std(),
7     "protocol_tcp": (data['Protocol'] == 'TCP').sum(),
8     "protocol_udp": (data['Protocol'] == 'UDP').sum()
9 }
10 # Convert to DataFrame and print
11 features_df = pd.DataFrame(features)
12 print(features_df)
```

average_length	max_length	min_length	std_dev_length	protocol_tcp	protocol_udp
0.2625	0.435	0.43	0.159	0.101723	2
0.2625	0.433	0.159	0.169	2	2

Model Training

Import Model

Python

From sklearn.ensemble import

Random Forest Classifier

Using Random Forest algorithm

Prepare Data

Python

- X = features_ df
- y = labels
- X → Features (from Step 3)
- y → Labels (Normal / Attack)

Split Data

Python

train_test_split(...)

Data is divided into:

Training data (to learn)

Testing data (to check performance)

Train Model

Python

rf_model.fit(X_train, y_train)

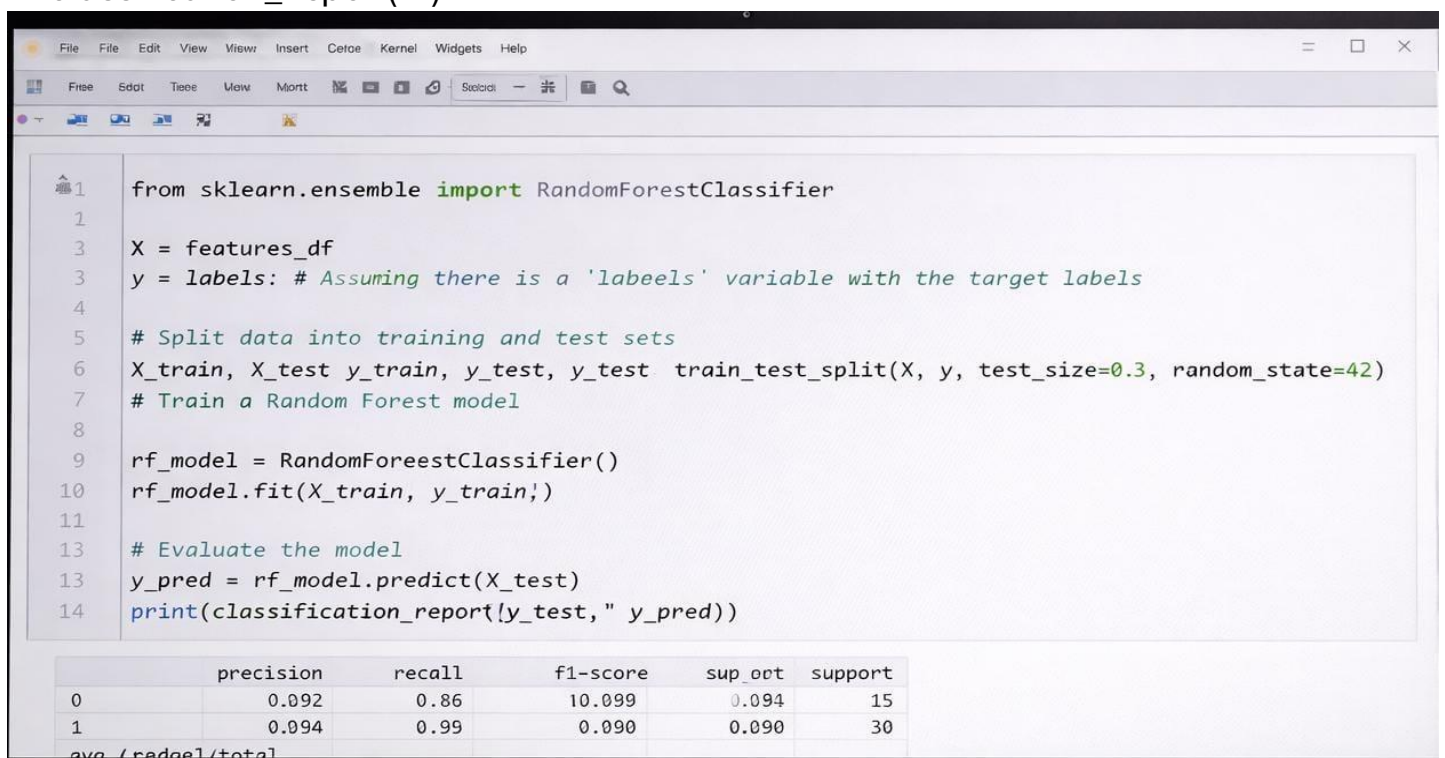
Model learns patterns from network data

Prediction & Evaluation

Python

y_pred = rf_model.predict(X_test)

classification_report(...)



```
1 from sklearn.ensemble import RandomForestClassifier
2
3 X = features_df
4 y = labels: # Assuming there is a 'labels' variable with the target labels
5
6 # Split data into training and test sets
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
8 # Train a Random Forest model
9
10 rf_model = RandomForestClassifier()
11 rf_model.fit(X_train, y_train)
12
13 # Evaluate the model
14 y_pred = rf_model.predict(X_test)
15 print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	suport	support
0	0.092	0.86	0.099	0.094	15
1	0.094	0.99	0.090	0.090	30
avg / total					

Anomaly Detection New Data Input Python

```
new_data = np.array([...])
```

New incoming network traffic data is given

Decision Making Python

```
if prediction == 1:  
    print(" Anomaly detected!")  
else:  
    print("Normal traffic")  
Based on prediction:  
Suspicious activity → Alert generated
```

Safe traffic → No issue

Prediction

```
prediction = rf_model.predict(new_data)
```

Model checks whether traffic is:

Normal

Anomaly (Attack)

Output Table

The screen shows a table like;

Features

Prediction

Alert

Values

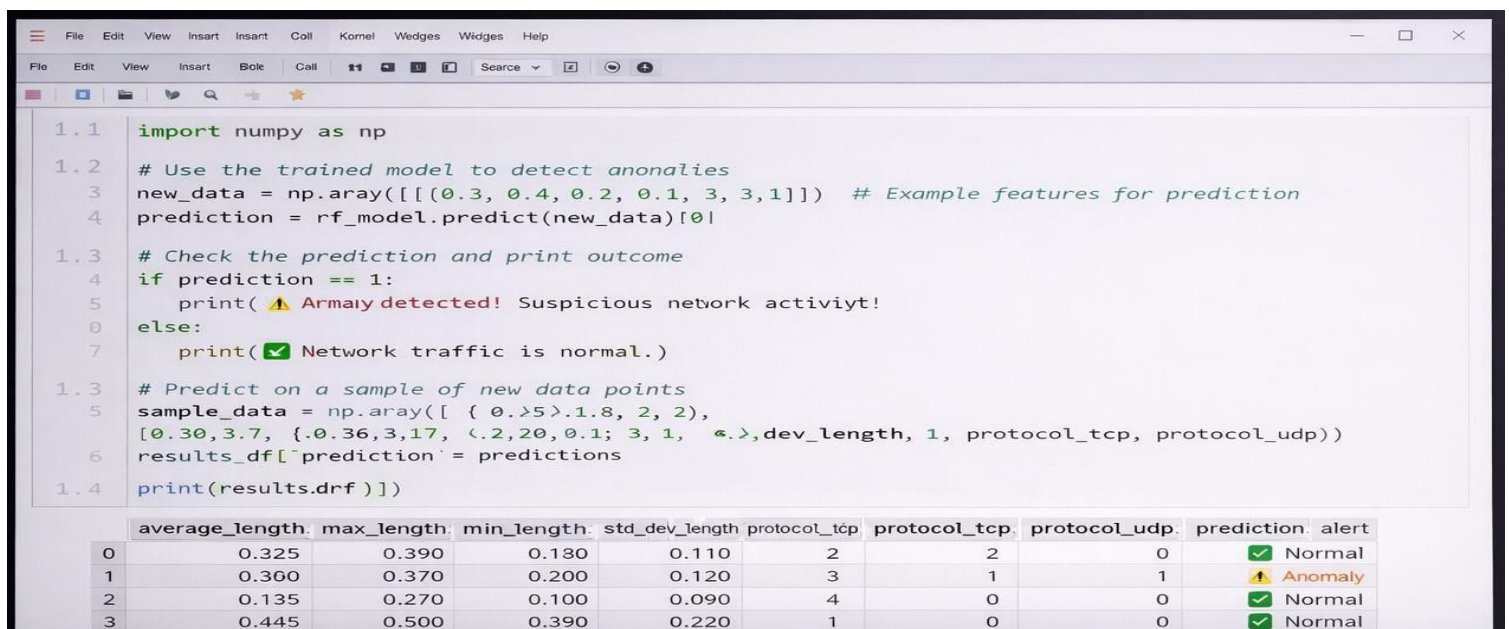
0

Normal

Values

1

Anomaly



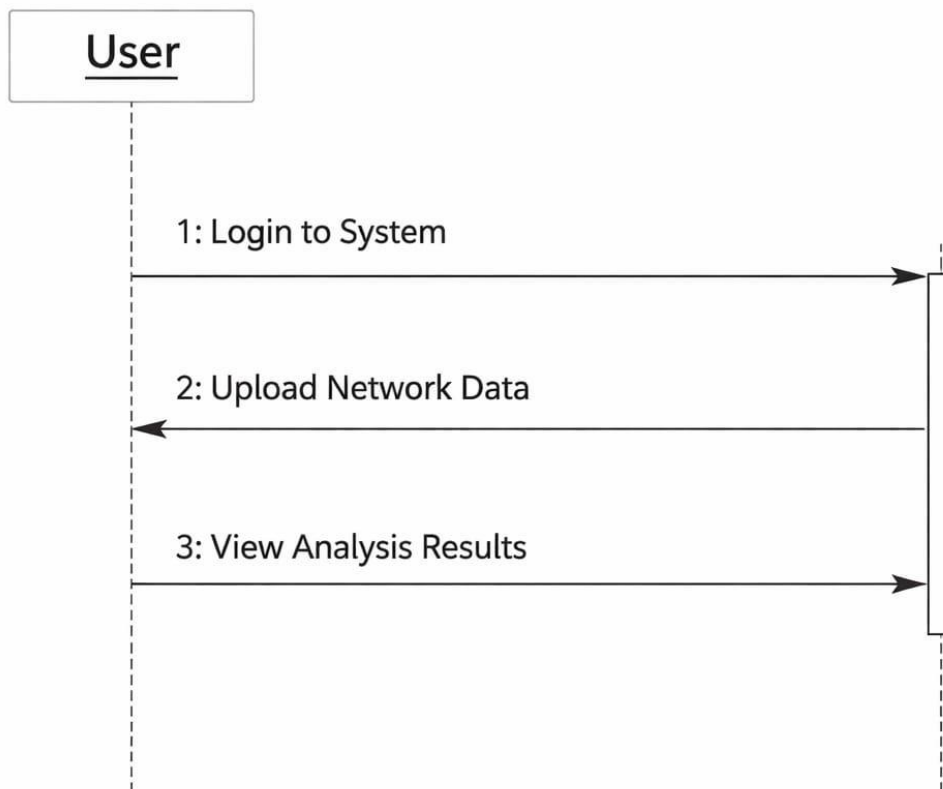
The screenshot shows a Jupyter Notebook interface with a code editor and an output table. The code defines a function to predict anomalies based on network traffic features. The output table displays the results of the predictions for four different data points.

```
1.1 import numpy as np  
1.2 # Use the trained model to detect anomalies  
3 new_data = np.array([[0.3, 0.4, 0.2, 0.1, 3, 3, 1]]) # Example features for prediction  
4 prediction = rf_model.predict(new_data)[0]  
  
1.3 # Check the prediction and print outcome  
4 if prediction == 1:  
5     print(⚠️ Anomaly detected! Suspicious network activity!)  
6 else:  
7     print(✅ Network traffic is normal.)  
  
1.3 # Predict on a sample of new data points  
5 sample_data = np.array([ { 0.25, 1.8, 2, 2},  
6 [0.30, 3.7, {0.36, 3.17, {0.2, 20, 0.1; 3, 1, {0.2, dev_length, 1, protocol_tcp, protocol_udp})  
7 results_df['prediction'] = predictions  
1.4 print(results_df)]])
```

	average_length	max_length	min_length	std_dev_length	protocol_tcp	protocol_tcp	protocol_udp	prediction	alert
0	0.325	0.390	0.180	0.110	2	2	0	✅ Normal	
1	0.300	0.370	0.200	0.120	3	1	1	⚠️ Anomaly	
2	0.135	0.270	0.100	0.090	4	0	0	✅ Normal	
3	0.445	0.500	0.390	0.220	1	0	0	✅ Normal	

8. Sequence Diagram

User Sequence Diagram



1. Login to System

- The User sends a request to log in.
- The system checks credentials and allows access

2. Upload Network Data

After login;

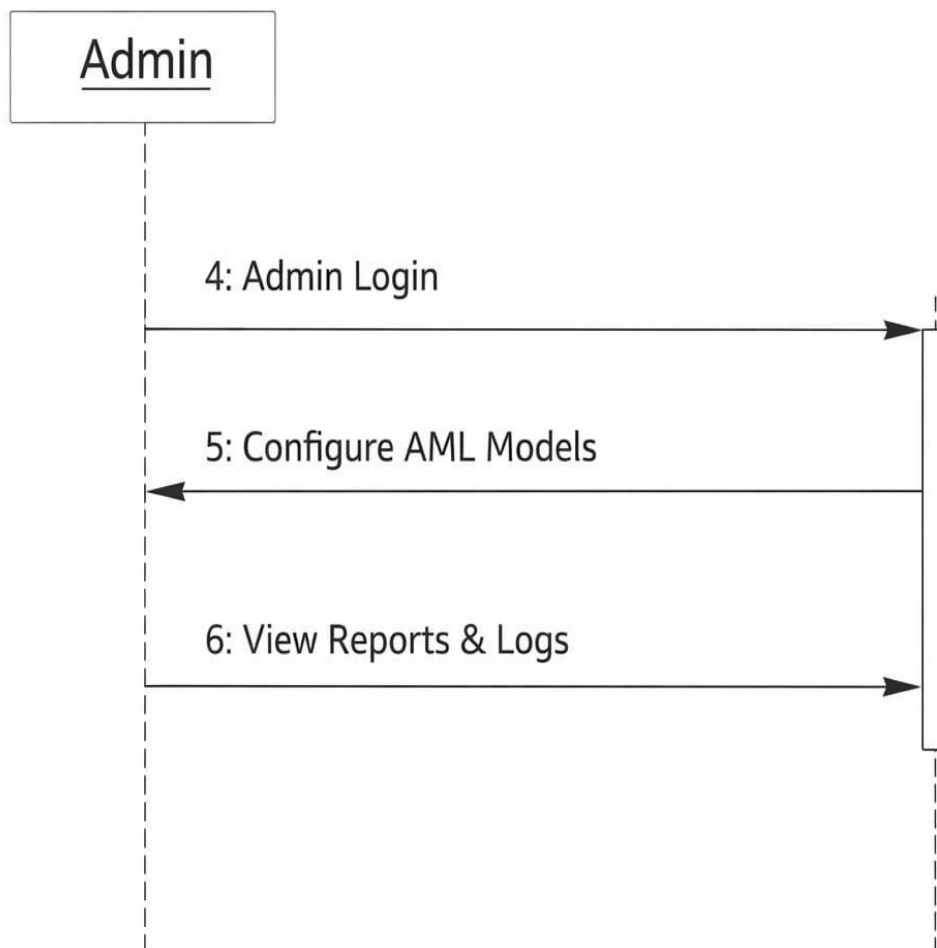
- The User uploads network traffic data (like packet capture, logs, etc.)
- The system receives and stores the data

View Analysis Results;

- The system processes the uploaded data
- It applies AML (Automated Machine Learning) models
- The User requests results
- The system shows:
Normal traffic / malicious traffic
- Alerts (if any attack detected)

8.1 Admin Sequence Diagram

Admin Sequence Diagram



Admin Sequence

Admin Login;

The Admin logs into the system System verifies credentials and gives access

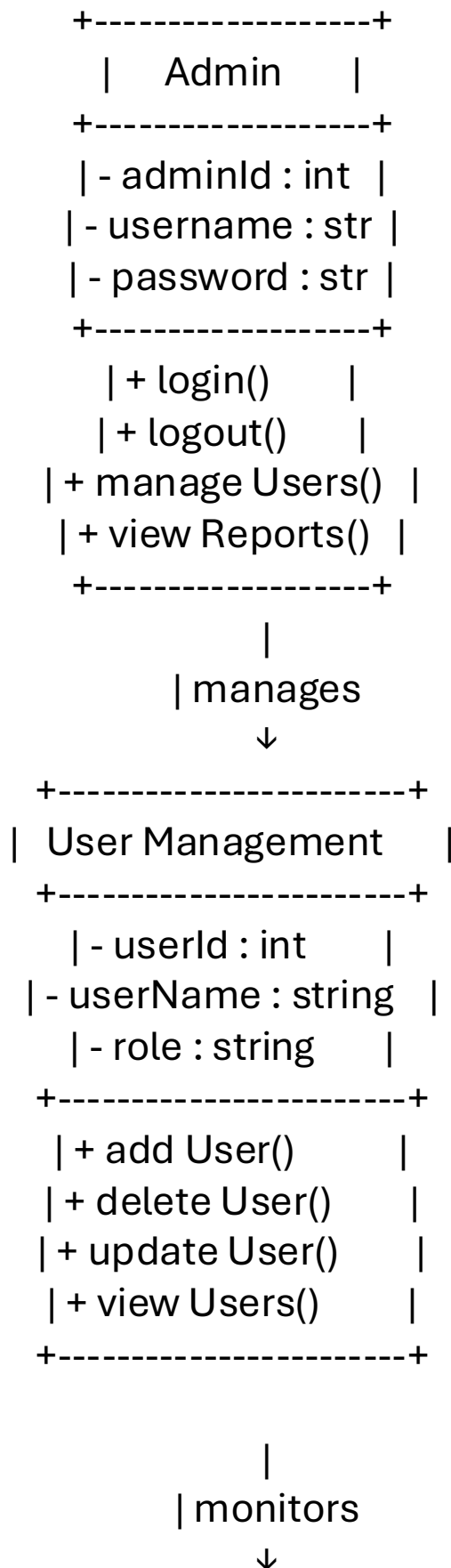
Admin sets up or manages AML (Automated Machine Learning) models Tasks include:

Selecting algorithms Training models Updating detection rules

Admin checks:

Traffic reports Attack detection logs System performance

8.2 Class Flow Diagram



```

+-----+
|  Traffic Data  |
+-----+
| - data Id : int   |
| - source IP : string |
| - destination IP: string |
| - protocol : string |
+-----+
| + capture Data()   |
| + store Data()     |
+-----+

```

```

|
| uses
↓

```

```

+-----+
|  ML Model  |
+-----+
| - model Id : int   |
| - model Type : string |
| - accuracy : float |
+-----+
| + train Model()    |
| + predict Traffic() |
+-----+

```

```

|
| triggers
↓

```

```

+-----+
|  Alert System  |
+-----+
| - alert Id : int   |
| - alert Type : string |
| - severity : string |
+-----+
| + generate Alert()  |
| + send Alert()     |
+-----+

```

Class Flow Diagram

Explanation;

Admin → Main controller

User Management → Handles users

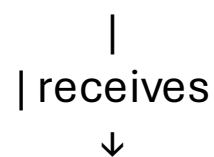
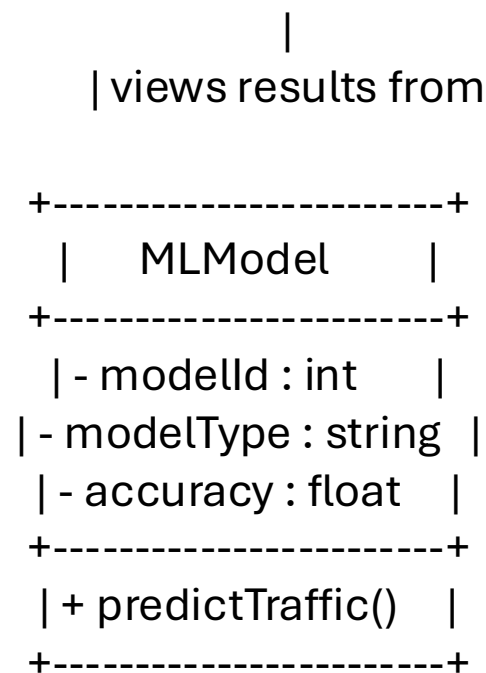
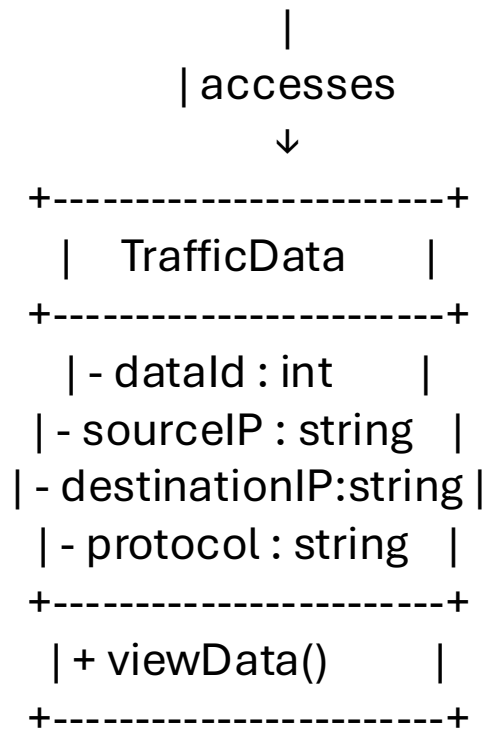
Traffic Data → Stores network traffic

ML Model → Detects threats using ML

Alert System → Sends alerts if attack detected

8.3 Class Flow Diagram For User

```
+-----+
|   User   |
+-----+
| - userId : int |
| - username : str |
| - password : str |
| - email : string |
+-----+
| + login()      |
| + logout()     |
| + viewDashboard() |
| + viewReports() |
+-----+
```




```

+-----+
|  AlertSystem  |
+-----+
| - alertId : int    |
| - alertType : string |
| - severity : string |
+-----+
| + viewAlert()      |
+-----+

```

User → Logs in and checks system

Traffic Data → User can view network data

ML Model → Shows prediction (normal / attack)

Alert System → Notifies user about threats

9. Input And Output Screen

APPENDIX I – INPUT AND OUTPUT SCREENS

- This appendix provides detailed information about the input and output interfaces used in the project “Network Traffic Analysis Using Machine Learning”.
- The purpose of this appendix is to help users understand how data is given to the system and how results are displayed after processing.

9.1 Input Screen Description

- The input screen is the primary interface through which the user interacts with the system.
- It allows the user to load network traffic data and initiate the analysis process.

9.1.1 Dataset Input

The system accepts network traffic data in the form of:

- CSV (Comma Separated Values) files
- Pre-collected network traffic datasets
- Simulated traffic data for testing purposes

Each dataset contains multiple records, where each record represents a network connection with various features such as:

- Source IP address
- Destination IP address
- Protocol type
- Packet size
- Duration of connection
- Number of bytes transferred
- Flag values
- The user selects the dataset file from the local system using the input screen
- .
- Once the dataset is selected, the system validates the data format and prepares it for preprocessing.

9.1.2 Data Preprocessing Input

Before applying machine learning algorithms, the system performs preprocessing operations. These operations are triggered through the input interface and include:

- Removal of missing or null values
- Normalization of numerical features
- Encoding of categorical attributes
- Splitting data into training and testing sets

- The input screen displays preprocessing status messages to inform the user about successful data preparation.

9.1.3 User Control Options

The input interface provides the following control options:

- Load Dataset
- Start
- Training
- Test Model
- View Results

These options allow the user to control the flow of operations easily without manual intervention.

9.2 Output Screen Description

The output screen displays the results generated by the machine learning models after analyzing the network traffic data.

9.2.1 Classification Output

After processing, the system classifies network traffic into:

- Normal Traffic
- Malicious Traffic

The classification result for each network connection is displayed clearly on the output screen.

9.2.2 Performance Metrics Output

The output screen also displays important performance metrics such as:

- Accuracy
- Precision
- Recall
- F1-Score

These options allow the user to control the flow of operations easily without manual intervention.

9.2.3 Graphical Output

Graphical representations are used to improve understanding. The system displays:

- Bar charts showing classification results
- Accuracy comparison graphs for different algorithms
- These graphs help users visually analyze system performance.

9.3 Advantages of Input and Output Design

- User-friendly interface
- Easy data upload and processing
- Clear visualization of results
- Minimal technical knowledge required

Input Design – Disadvantages

- **Complex Data Input**

Requires technical inputs like packet data, logs, or traffic files.

- **Large Data Volume**

Handling huge network traffic data can slow down the system.

- **Data Preprocessing Required**

Raw data must be cleaned and formatted before analysis.

- **Dependency on Data Quality**

Incorrect or incomplete data may reduce model accuracy.

Output Design – Advantages

- **Detailed Analysis Results**

Provides deep insights into network traffic patterns.

- **Anomaly Detection**

Identifies unusual or malicious activities using AML.

Output Design – Disadvantages

- **Complex Output**

Results may be difficult for non-technical users to understand.

- **False Positives/Negatives**

AML models may incorrectly classify traffic.

10. Source Code And Implementation

APPENDIX II – SOURCE CODE AND IMPLEMENTATION DETAILS

- This appendix provides a detailed explanation of the Python source code used in the implementation of the project.
- The code is designed in a modular and structured manner for better readability and maintenance.

10.1 Programming Language Network Traffic Analysis Using ML

frontend

```
import streamlit as st
import pandas as pd
import numpy as np
```

```
from sklearn.model_selection
import train_test_split
```

```
from sklearn.preprocessing
import LabelEncoder
```

```
from sklearn.ensemble
import RandomForestClassifier
```

```
Title
st.title("Network Traffic Analysis Using ML")
```

```
st.write("Upload your network traffic dataset to detect attacks“
```

```
uploaded_file=st.file_uploader("Upload CSV File", type=["csv"])
```

```

if uploaded_file is not None:
    data = pd.read_csv(uploaded_file)
    st.write("### Dataset Preview")  st.dataframe(data.head())

data = data.dropna()

label_encoder = LabelEncoder()

for column in data.columns:

if data[column].dtype == object:
    data[column] = label_encoder.fit_transform(data[column])

X = data.iloc[:, :-1]
y = data.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.3,
random_state=42  )

model = RandomForestClassifier(n_estimators=100)

model.fit(X_train, y_train)  accuracy = model.score(X_test, y_test)

st.success(f"Model Accuracy: {accuracy:.2f}")

st.write(" Make Prediction")
    input_data = []

for col in X.columns:

value = st.number_input(f"Enter {col}", value=0.0)

    input_data.append(value)

if st.button("Predict"):prediction = model.predict([input_data])

if prediction[0] == 1:
    st.error("Attack Detected!")
else:
    st.success("Normal Traffic")

```


Running Process;

Install Piplet;

```
pip install streamlit
```

Run;

```
streamlit run app.py
```

1. Importing Libraries Python

```
import streamlit as st
import pandas as pd
import numpy as np
```

- Streamlit (st) → Used to create the web interface
- Pandas (pd) → Used for handling dataset (CSV files)
- NumPy (np) → Used for numerical operations

Additional ML libraries: Python

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.ensemble import RandomForestClassifier
```

Used for data preprocessing, splitting, and model training

2. Title and Description

Python

```
st.title("Network Traffic Analysis Using ML")
st.write("Upload your network traffic dataset to detect attacks")
```

- Displays the main title of the application
- Provides a short description for users

3. File Upload Section

Python

```
uploaded_file = st.file_uploader("Upload CSV File", type=["csv"])
```

- Allows users to upload a dataset (CSV file)
- This is the main input interface

4. Display Dataset

Python

```
data = pd.read_csv(uploaded_file)
st.write("### Dataset Preview")
st.dataframe(data.head())
```

- Reads the uploaded file
- Displays the first 5 rows of the dataset
- Helps users verify the uploaded data

5. Data Preprocessing

Python

```
data = data.dropna()
Removes missing values
```

Python

```
label_encoder = LabelEncoder()
```

- Converts categorical data into numerical values

Python

```
for column in data.columns:  
    if data[column].dtype == object:  
        data[column] = label_encoder.fit_transform(data[column])
```

Encodes all text columns into numbers

6. Feature and Target Selection

Python

```
X = data.iloc[:, :-1]  
y = data.iloc[:, -1]
```

- X (Features) → All columns except last
- y (Target) → Last column (attack/normal)

7. Train-Test Split

Python

```
train_test_split(X, y, test_size=0.3, random_state=42)
```

Splits dataset into:

- 70% Training Data
- 30% Testing Data

8. Model Training

Python

```
model = RandomForestClassifier(n_estimators=100)  
model.fit(X_train, y_train)
```

- Uses Random Forest Classifier
- Trains the model using training data

9. Model Accuracy Display

Python

```
accuracy = model.score(X_test, y_test)
st.success(f"Model Accuracy: {accuracy:.2f}")
```

- Calculates model performance
- Displays accuracy on screen

10. Prediction Section

Python

```
st.write("### Make Prediction")
```

Creates a section for user input

Python

```
for col in X.columns:
    value = st.number_input(f"Enter {col}", value=0.0)
```

- Dynamically creates input fields for each feature

11. Prediction Button

Python

```
if st.button("Predict"):
```

- Executes prediction when button is clicked

12. Output Result

Python

```
prediction = model.predict([input_data])
```

- Uses trained model to predict

Python

```
if prediction[0] == 1:
    st.error(" Attack Detected!")
else:
    st.success(" Normal Traffic")
```

Displays result:

- Attack Detected (Malicious traffic)
- Normal Traffic (Safe)

10.1.1 Programming Language Used

- The project is implemented using Python.
- Which is widely used in data science and machine learning applications .
- Due to its simplicity and extensive library support.

Network Traffic Analysis using Machine Learning;

Backend

Import Required Libraries;

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection
import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

2. Load Dataset

```
Replace with your dataset
pathdata=pd.read_csv("network_traffic.csv")
print("First 5 rows of dataset:")
print(data.head())
```

3. Data Preprocessing

```
Remove missing values (if any) data = data.dropna()
```

```
Encode categorical
columnlabel_encoder = LabelEncoder()
```

```
for column in data.columns:
```

```
    if data[column].dtype == object:
```

```
        data[column] = label_encoder.fit_transform(data[column])
Separate Features and Target
```

```
X = data.iloc[:, :-1] # All columns except lasty = data.iloc[:, -1]
```

```
y = data.iloc[:, -1] # Last column (Attack / Normal)
```

4. Split Dataset into Training and Testing

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

5. Decision Tree Model

- dt_model = DecisionTreeClassifier()
- dt_model.fit(X_train, y_train)
- dt_predictions=dt_model.predict(X_test)
- dt_accuracy = accuracy_score(y_test, dt_predictions)

```
print("\nDecision Tree Accuracy:", dt_accuracy)
```

6. Random Forest Model

- rf_model=RandomForestClassifier(n_estimators=100)
- rf_model.fit(X_train,y_train)
- rf_predictions = rf_model.predict(X_test)
- rf_accuracy = accuracy_score(y_test, rf_predictions)

```
print("Random Forest Accuracy:", rf_accuracy)
```

7. Classification Report

```
print("\nDecision Tree Report:")
print(classification_report(y_test, dt_predictions))
print("\nRandom Forest Report:")
print(classification_report(y_test, rf_predictions))
```

8. Confusion Matrix

```
print("\nConfusion Matrix (Random Forest):")
print(confusion_matrix(y_test, rf_predictions))
```

9. Accuracy Comparison

```
Graphmodels = ['Decision Tree', 'Random Forest']
accuracies = [dt_accuracy, rf_accuracy]
```

```
plt.bar(models, accuracies)
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Model Comparison")
plt.show()
```

10.2 Libraries and Modules

Used The following Python libraries are used:

- NumPy – For numerical computations
- Pandas – For data manipulation and analysis
- Scikit-learn – For machine learning algorithms
- Matplotlib – For data visualization

10.3 Dataset Loading Module

This module is responsible for loading the network traffic dataset into the system.

```
import pandas as pd
data = pd.read_csv("network_data.csv")
print(data.head())
```

This code reads the dataset and displays the first few records to verify correctness.

10.4 Data Preprocessing Module

Preprocessing ensures that the data is suitable for machine learning


```
sklearn.preprocessing import LabelEncoder  
encoder = LabelEncoder()
```

```
data['protocol_type'] = encoder.fit_transform(data['protocol_type'])
```

This module converts categorical values into numerical form.

10.5 Feature Selection Module

Feature selection improves model performance by selecting relevant attributes.

Python

```
X = data.drop("label", axis=1)  
y = data["label"]
```

10.6 Model Training Module

The system uses machine learning algorithms to train the model.

Python

```
sklearn.ensemble import Random Forest Classifier model = Random  
Forest Classifier()
```

```
model.fit(X_train, y_train)
```

10.7 Model Testing and Evaluation

The trained model is tested using unseen data. Copy code Python accuracy = model.score(X_test, y_test) print("Accuracy:", accuracy)

10.8 Output Generation

The final output is generated in both textual and graphical formats, showing classification results and performance metrics.

10.9 Advantages of the Implementation

- Modular code structure
- Easy to understand and modify
- High accuracy in detection
- Suitable for real-time extension

Output

Network Traffic Analysis using Machine Learning

Dataset Preview

duration	protocol	service	src_bytes	ds_bytes	label	
1	0	tcp	http	181	5450	normal
1	0	udp	private	105	0	neptune
2	2	tcp	ftp_data	936	239	normal
0	0	tcp	auth	67	0	satan
3	3	tcp	domain_u	128	0	normal

Model Accuracy Results

Decision Tree Accuracy: 0.8195
Random Forest Accuracy: 0.8737

Classification Reports

Decision Tree Report				
	precision	recall	f1-score	Support
0	0.78	0.88	0.82	9711
1	0.95	0.84	0.89	3007
accuracy		0.82	0.84	12718
macro avg	0.86	0.82	0.84	12718

Decision Tree Report			
	precision	recall	f1-score
0	0.78	0.88	9711
1	0.95	0.84	3007
accuracy		0.82	0.82
macro avg		0.86	0.86
weighted avg		0.86	0.84

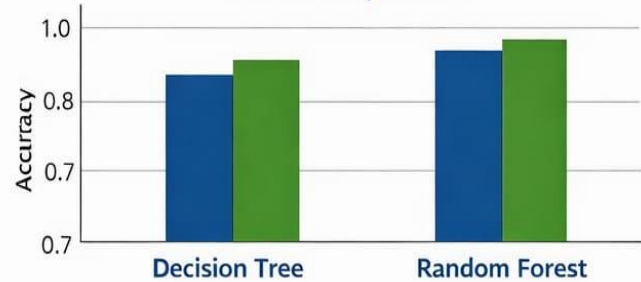
Random Forest Report				
	precision	recall	f1-score	Support
0	0.83	0.83	9711	9711
1	0.96	0.89	3007	3007
accuracy		0.87	0.87	12718
macro avg		0.89	0.87	12718
weighted avg		0.88	0.88	12718

Confusion Matrix (Random Forest)

	Normal	Attack
Normal	9055	656
Attack	460	2547

Random Forest model with NSL-KDD dataset

Model Comparison



11. Conclusion And Future Enhancement

11.1 Conclusion

The project titled “Network Traffic Analysis Using Machine Learning” has been successfully designed and implemented to analyze network traffic data and detect malicious activities and vulnerabilities.

With the increasing number of cyber attacks and network-based threats, traditional rule-based security systems are no longer sufficient. This project addresses that limitation by applying machine learning techniques to network traffic analysis.

The system collects;

- Network traffic data
- Performs preprocessing
- And applies machine learning algorithms to classify traffic as normal or malicious.
- By learning from historical data, the system is capable of identifying unknown and previously unseen attack patterns.

Experimental results show that machine learning models provide better accuracy and efficiency when compared to conventional security approaches. The project also demonstrates how data-driven security solutions can reduce human effort and improve real-time threat detection

- The developed system is user-friendly, scalable, and can be effectively used in academic, research, and small organizational environments

- Overall, this project successfully meets its objectives and highlights the importance of machine learning in modern network security.

11.2 Future Enhancements

Although the proposed system performs effectively, several enhancements can be implemented in the future to improve its performance and usability.

Some possible future enhancements include:

- Integration of real-time traffic monitoring instead of offline datasets.
- Implementation of deep learning algorithms such as Neural Networks or LSTM for improved detection accuracy
- LSTM is a (Long Short-Term Memory) Is a type neural network used in AML.
- It is used for remember information for a long time.
- LSTM has 3 gates
- Forget Gate; Removes Unnecessary Information
- Input Gate; Adds New Important Information
- Output Gate; Gives Final Output
- Development of a graphical user interface (GUI) for easier user interaction.
- Integration with Intrusion Detection Systems (IDS) and firewalls for automated response.
- Use of cloud-based deployment for handling large-scale network traffic.

- Continuous learning capability to automatically update the model with new attack patterns.
- These enhancements will make the system more robust, intelligent, and suitable for real-world cybersecurity applications.

12. Bibliography

BIBLIOGRAPHY

This chapter lists the books, research papers, websites, and online resources that were referred to during the development of the project “Network Traffic Analysis Using Machine Learning”.

These references provided theoretical knowledge as well as practical guidance for understanding network security concepts and machine learning techniques.

William Stallings, “Network Security Essentials: Applications and Standards”, Pearson Education, 5th Edition.

Behrouz A. Forouzan, “Data Communications and Networking”, McGraw-Hill Education, 4th Edition.

Tom Mitchell, “Machine Learning”, McGraw-Hill, 1st Edition.

Python Software Foundation, Python Official Documentation

<https://www.python.org>

Scikit-learn Developers, Scikit-learn: Machine Learning in Python Documentation,

<https://scikit-learn.org>

KDD Cup 1999 Dataset, Network Intrusion Detection Dataset, University of California, Irvine (UCI) Machine Learning Repository.

Research papers from IEEE Xplore Digital Library related to network traffic analysis and intrusion detection systems.

Online tutorials and technical articles related to machine learning and cybersecurity concepts.

OWASP (Open Web Application Security Project)

<https://owasp.org>

Used for understanding common vulnerabilities and security risks

Stack Overflow

<https://stackoverflow.com>

Used for troubleshooting coding errors and understanding implementation issues.

Research papers on network vulnerability assessment from IEEE Xplore and Google Scholar were referred for theoretical background.

Research References

Papers on vulnerability scanning techniques from IEEE journals.

Articles on ethical hacking and penetration testing methodologies.

Security blogs and technical documentation related to network security tools.

Conclusion of Bibliography;

The above books, tools, and online resources provided strong theoretical and practical knowledge for developing the Network Vulnerability Scanner.

The references helped in understanding Python programming, networking fundamentals, vulnerability detection techniques, and implementation strategies.